

STYLE360 BACKEND VIVA MASTER GUIDE

Style360 ව්‍යාපෘතියේ Backend Architecture, PHP APIs, AI Orchestration, Core Algorithms සහ MySQL Database සම්බන්ධව Viva පරීක්ෂණයේදී අසනු ලබන සියලුම ප්‍රශ්න හා කේත කොටස් පිළිබඳ විශේෂ සිංහල මාර්ගෝපදේශය.

BACKEND ARCHITECTURE

PHP 8.2 + MySQL

Stateless RESTful APIs with
PDO connection resilience &
zero memory leaks.

AI MICROSERVICES

Fal.ai + Tripo3D

Fashn v1.6 Virtual Try-On,
Tripo3D V3 Image-to-3D &
Gemini Flash Vision.

SPECIAL FEATURES

Caching & Proxy

MD5 Mesh Caching, GD
Resampling & CORS-free
GLB Binary Streaming.

Candidate Role: Backend Developer & Systems Integrator

Project: Style360 - AI Powered Virtual Fitting & 3D Fashion Platform

Framework & Tools: PHP 8.2 • MySQL 8.0 • PDO • Three.js GLB Proxy •
Google Gemini Vision

Academic Defense
Edition
September 2026

01 Backend System Architecture & Technology Stack

Style360 ව්‍යාපෘතියේ Backend එක නිර්මාණය කර ඇත්තේ **Stateless RESTful API Architecture** එකක් ලෙසිනි. Frontend එකෙහි (React 18 + Vite) සිට පැමිණෙන පරිශීලක ඉල්ලීම් (HTTP Requests) භාරගෙන, දත්ත සම්ප්‍රදාය (MySQL) සමඟ ගනුදෙනු කරමින්, කෘතීම බුද්ධි සේවාවන් (Fal.ai, Tripo3D, Google Gemini) සම්බන්ධීකරණය (orchestrate) කිරීම Backend හි ප්‍රධාන කාර්යභාරයයි.

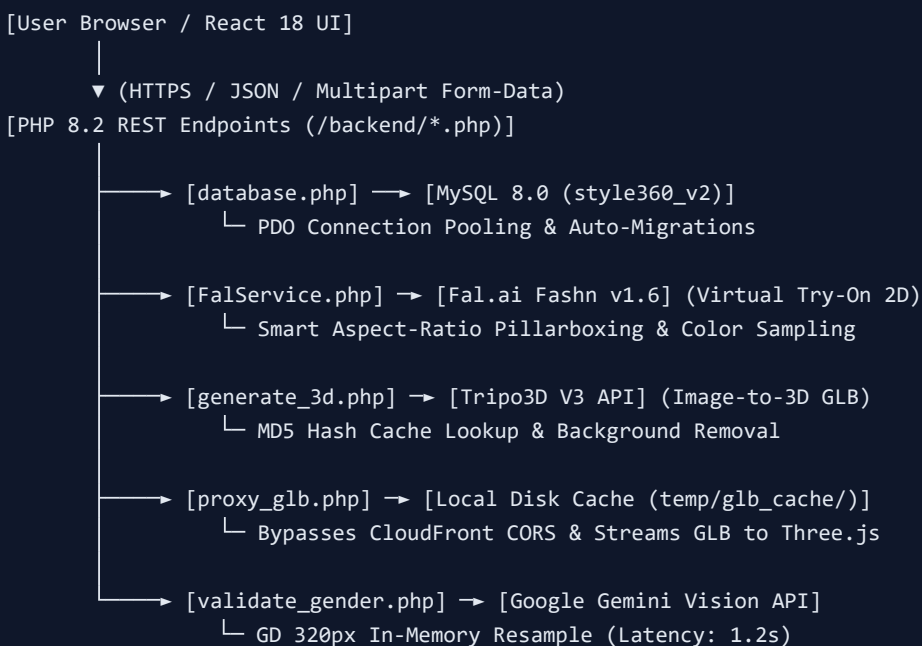
💡 Why PHP 8.2 instead of Node.js or Python for Backend? (Viva Defense Key Point)

විවා පරීක්ෂකවරුන් නිතර අසන ප්‍රශ්නයකි: "AI project එකකට Python හෝ Node.js භාවිතා නොකර PHP තෝරාගත්තේ ඇයි?"

පිළිතුර:

- **Stateless Process Isolation (මනක කාන්දු වැළැක්වීම):** 3D GLB Models (5MB - 15MB) Binary Streaming කිරීමේදී Node.js වැනි single-threaded event loop එකක RAM Buffer කාන්දු (Memory Leaks) ඇති විය හැක. PHP හි සෑම request එකක්ම වෙනම process එකක් ලෙස ක්‍රියාත්මක වී අවසානයේ memory සම්පූර්ණයෙන්ම නිදහස් (garbage collected) කෙරේ.
- **Zero Cold-Start Latency:** Python Web Frameworks (FastAPI, Django) මෙන් නොව, PHP Apache/Nginx මත ඉතා වේගයෙන් (<15ms) initial request cycle එක ආරම්භ කරයි.
- **Built-in GD Image Library & Stream Wrappers:** තෙවන පාර්ශවීය බැරැති packages නොමැතිව Native GD මඟින් High-Speed Image Resampling සහ cURL මඟින් Multi-Part Cloud API streaming සිදුකළ හැක.
- **Cost & Portability:** ඕනෑම Standard Cloud VM එකක (Shared, VPS, Dedicated) ඉතා අඩු වියදමකින් සහ පහසුවෙන් deploy කළ හැක.

Backend High-Level Data Flow



Backend Core Files - Detailed Code & Logic

Breakdown

Style360 Backend එකෙහි අඩංගු සෑම file එකකම කාර්යභාරය, එහි ඇති විශේෂ Functions, Algorithms සහ Security Measures පහත පරිදි අධ්‍යයනය කර Viva එකේදී පැහැදිලි කරන්න:

backend/database.php

DATABASE & MIGRATIONS

මූලික කාර්යය: MySQL දත්ත සමුදාය සමඟ PDO සම්බන්ධතාවය ඇති කිරීම, පරිසර විචල්‍යයන් (.env) කියවීම, සහ දත්ත සමුදාය හෝ වගු (Tables) නොමැති නම් ඒවා ස්වයංක්‍රීයව නිර්මාණය කිරීම (Auto-Provisioning & Auto-Migration).

විශේෂ Code කොටස් සහ Logic:

- 1. Zero-Dependency .env Parser:** Composer හෝ vlucas/phpdotenv වැනි external packages මත යැපෙන්නේ නැතිව Native PHP string operations මගින් root `.env` file එක කියවා `$_ENV` සහ `putenv()` වෙත load කරයි:

```
if (file_exists($envPath)) {
    $lines = file($envPath, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    foreach ($lines as $line) {
        if ($line === '' || strpos($line, '#') === 0) continue;
        if (strpos($line, '=') !== false) {
            list($key, $val) = explode('=', $line, 2);
            putenv(trim($key) . "=" . trim($val, " \t\n\r\0\x0B\""));
        }
    }
}
```

- 2. Host Fallback Array & Auto-Creation:** Localhost, 127.0.0.1, custom port ආදී විවිධ පරිසරයන්හිදී සම්බන්ධතාවය බිඳ නොවැටී පවත්වා ගැනීමට `$hostsToTry` array එකක් භාවිත කරයි. `style360_v2` database එක නොමැති නම් root මගින් `CREATE DATABASE IF NOT EXISTS` execute කර auto-create කරයි.
- 3. Live Schema Migrations & Data Cleaning:** `SHOW COLUMNS` පරීක්ෂා කර `result_image_url` වැනි අලුත් columns එකතු කිරීම, CloudFront signed URLs (>900 chars) ගබඩා කිරීම සඳහා `result_3d_url TEXT` වෙට් පත් කිරීම, සහ static mock dummy images database එකෙන් ස්වයංක්‍රීයව delete කර දත්තවල පිරිසිදුකම (Data Integrity) ආරක්ෂා කිරීම.

මූලික කාර්යය: Fal.ai Cloud Platform එකෙහි ඇති Fashn v1.6 Virtual Try-On සහ Flux Inpainting APIs කළමනාකරණය කරන මධ්‍යම Service Controller එකයි.

විශේෂ Code කොටස් සහ Logic:

- **Smart Aspect-Ratio Standardizer (Pillarboxing Algorithm):** පරිශීලකයා උඩුගත කරන ඡායාරූප (විශේෂයෙන් සම්පූර්ණ උස 9:16 portraits) AI Try-On එකට යැවීමේදී ඔලුව හෝ දෙපා කැපී යාම (crop වීම) වැළැක්වීමට මෙම ඇල්ගොරිතමය ක්‍රියාත්මක වේ. `standardizeModelImage()` මගින් මුළු උසම (100% vertical height) ආරක්ෂා කරමින් දෙපසට seamless padding එකතු කරයි.

```
// ඡායාරූපයේ කොනක (Corner) pixel වර්ණය හඳුනාගෙන canvas එක fill කිරීම:  
$rgb = @imagecolorat($srcImg, min(4, $origW - 1), min(4, $origH - 1));  
$r = ($rgb >> 16) & 0xFF; $g = ($rgb >> 8) & 0xFF; $b = $rgb & 0xFF;  
$bg = imagecolorallocate($canvas, $r, $g, $b);  
imagefilledrectangle($canvas, 0, 0, $targetW, $targetH, $bg);  
imagecopyresampled($canvas, $srcImg, $dstX, $dstY, 0, 0, $dstW, $dstH, $origW, $origH);
```

- **Face Restoration Weight Constraint (0.5 Constraint):** සාමාන්‍යයෙන් CodeFormer face restoration එක 1.0 අගයකදී ක්‍රියාත්මක වීමේදී හිස පමණක් කෘතීමව විශාල වී (oversized cartoonish head) දිස්වේ. Style360 හිදී මෙය `const DEFAULT_FACE_RESTORATION_WEIGHT = 0.5;` ලෙස සකසා ස්වභාවික මිනිස් සමානුපාතික බව (natural body-to-head proportions) තහවුරු කර ඇත.
- **isFullOutfit() Auto-Detection:** Garment category සහ title එකෙහි *"suit, two-piece, set, wedding, gown"* වැනි වචන හඳුනාගෙන Fal.ai වෙත `category = "one-pieces"` ලෙස යවා උඩ සහ යට කොටස් දෙකම සම්පූර්ණයෙන් අඳිනු ලබයි.

මූලික කාර්යය: 2D Virtual Try-On ප්‍රතිඵල ඡායාරූපය ලබාගෙන, පසුබිම ඉවත් කර (Background Removal), Tripo3D V3 AI Engine එක මගින් 3D Mesh එකක් (.GLB format) බවට පත් කිරීම.

විශේෂ Code කොටස් සහ Logic:

1. **MD5 Image Hash Caching Algorithm (Cost & Time Optimization):** Tripo3D API එකට එකම ඡායාරූපය නැවත නැවත යැවීමෙන් මුදල්/Credits නාස්ති වීම සහ තත්පර 15ක් බලා සිටීම වැළැක්වීමට, උඩුගත කරන ඡායාරූපයේ Base64 binary එකෙන් `md5($cleanBase64)` hash එකක් සාදයි. මෙම hash එක `user_3d_models` වගුවේ තිබේ නම්, API call එකක් නොකර **මිලි තත්පර 30කින් (30ms)** Cache එකෙන් කෙලින්ම 3D Model එක ලබාදෙයි!

```
$imageHash = md5($cleanBase64);
$stmt = $conn->prepare("SELECT id, model_url FROM user_3d_models WHERE image_hash = :hash LIMIT 1");
$stmt->execute([':hash' => $imageHash]);
if ($cached = $stmt->fetch()) {
    echo json_encode(['status' => 'success', 'cached' => true, 'model_url' => '/backend/proxy_gltf/' . $cached['model_url']]);
    exit;
}
```

2. **Automated Background Stripping:** ඡායාරූපයේ පසුබිමේ ඇති බිත්ති, දොරවල් හෝ බිම Tripo3D මගින් 3D ඝන වස්තු ලෙස වැරදියට generate වීම වැළැක්වීමට, පළමුව Fal Background Removal මගින් subject එක පමණක් වෙන්කර Transparent PNG එකක් ලෙස Tripo3D වෙත යවයි.
3. **Asynchronous Polling Loop with Adaptive Sleep:** Tripo task එකක් create කළ පසු, උපරිම තත්පර 90ක කාලයක් ඇතුළත සෑම තත්පර 2කට වරක් status එක poll කර බලා `status === 'success'` වූ වහාම GLB URL එක ලබාගෙන MySQL database එකේ persist කරයි.

මූලික කාර්යය: Three.js Frontend Viewer එකට දුරස්ථ AWS CloudFront S3 endpoints වෙතින් GLB files බාගත කිරීමේදී ඇතිවන CORS ආරක්ෂක බාධක මගහැරීම සහ දේශීයව Cache කිරීම.

⚠ **Why is proxy_glb.php indispensable? (Examiner Question)**

ප්‍රශ්නය: "Frontend එකට කෙලින්ම Tripo3D දෙන CloudFront GLB URL එක Three.js GLTFLoader එකට දෙන්න බැරිද? Proxy එකක් හැදුවේ ඇයි?"

පිළිතුර (Viva Answer):

1. **CORS (Cross-Origin Resource Sharing) Restriction:** CloudFront S3 CDN එකෙන් `Access-Control-Allow-Origin: *` header එක නොඑවන විට, බ්‍රවුසරයේ WebGL ආරක්ෂක නීති නිසා 3D model එක render වීම block වේ. මෙම proxy එකෙන් එම header එක inject කරයි.
2. **URL Expiration (Signed URLs):** Tripo3D signed URLs පැය 24කින් expire වේ. Proxy එක මගින් මුල් වරට බාගත කරන binary file එක `temp/glb_cache/model_[md5].glb` ලෙස hard disk එකේ තැන්පත් කරන බැවින්, මාස ගණනකට පසුව වුවද model එක නැවත load වේ!
3. **Streaming Without Memory Leaks:** `fpasssthru($fp)` හෝ cURL buffer stream එකක් භාවිතා කරන නිසා 15MB file එකක් වුවද PHP Server RAM එකට load නොවී කෙලින්ම client බ්‍රවුසරයට stream වේ.

මූලික කාර්යය: පරිශීලකයා තෝරාගත් ඇඳුම (Garment Gender: Male / Female) සහ උඩුගත කරන ලද පුද්ගලයාගේ ඡායාරූපයේ ලිංගිකත්වය ගැලපේදැයි පරීක්ෂා කිරීම. වැරදි gender mismatch එකක් හමු වුවහොත් expensive GPU Try-On එකට යාමට පෙර එය වහාම නවතයි.

Tier දෙකකින් සමන්විත Ultra-Fast Architecture:

- **Tier 1 - Regex Pre-check (<1ms):** Upload කරන file name එකෙහි පැහැදිලි වචන (උදා: `men, boy, tuxedo, bride, gown, saree`) නිවේදැයි Regex මගින් ක්ෂණිකව පරීක්ෂා කරයි. කිසිදු AI call එකක් නොකර 0.5ms තුළ තීරණය ගනී.
- **Tier 2 - GD In-Memory Downsampling & Gemini Vision (~1.2s):** පරිශීලකයා 4K හෝ 1080p high-resolution ඡායාරූපයක් (5MB) උපුන් වීට, එය කෙලින්ම Gemini API එකට යැවුවහොත් network upload එකට තත්පර 5-7ක් ගතවේ. Style360 Backend එකෙහි PHP GD library මගින් RAM එක තුළදීම එම රූපය **320px කුඩා Thumbnail එකක් (~12KB)** බවට resample කරයි!

```
// 320px In-Memory Resampling - Latency drops from 6.0s to 1.2s:
$maxDim = 320;
$dst = imagecreatetruecolor($nw, $nh);
imagecopyresampled($dst, $src, 0, 0, 0, 0, $nw, $nh, $w, $h);
ob_start(); imagejpeg($dst, null, 75); $jpgBytes = ob_get_clean();
$b64Thumb = base64_encode($jpgBytes);
```

අනතුරුව `gemini-flash-lite-latest` වෙත මෙම 12KB payload එක යවා zero temperature යටතේ තනි වචනයකින් (male/female) ප්‍රතිඵලය ලබාගනී.

මූලික කාර්යය: පරිශීලකයා හෝ Admin විසින් Catalog එකට ඇතුළත් කරන ඇඳුම සැබවින්ම ඇඳුමක්ද (Garment), එය අදින ආකාරය කුමක්ද (top, bottom, full-body suit, dress), සහ එහි කිසියම් නුසුදුසු අන්තර්ගතයක් (inappropriate content) නිවේදැයි Gemini Vision මගින් තහවුරු කිරීම.

මූලික කාර්යය: ඇඳුම් නාමාවලියෙහි (Catalog) සියලුම CRUD (Create, Read, Update, Delete) මෙහෙයුම් සිදුකරයි. Dynamic SQL Query Builder එකක් මගින් Gender, Category, Status සහ Search Query අනුව තත්ත්වයෙන් සුළු කාලයකදී filter කර JSON ලබාදෙයි.

- **GET** `/backend/garments.php?gender=male&category=suits` : Active garments පෙළගස්වයි.
- **POST** `/backend/garments.php` : අලුත් garment එකක් upload කිරීම (Display Image සහ FAL Transparent Image වෙත වෙනම save වේ).
- **DELETE** `/backend/garments.php?id=12` : Garment එකක් ඉවත් කිරීම (Safe cascade delete).

මූලික කාර්යය: User Sign-Up, Sign-In, Role Authorization (admin vs user) සහ PHP Session State කළමනාකරණය කිරීම.

ආරක්ෂක ක්‍රමවේද (Security Best Practices):

- **Bcrypt Strong Hashing:** Passwords කිසි විටෙකත් plain text ලෙස ගබඩා නොකරයි.
`password_hash($password, PASSWORD_DEFAULT)` මගින් 60-character cryptographically secure Blowfish salt hash එකක් සාදයි. Verify කිරීම සඳහා `password_verify()` භාවිත කරයි.
- **Complete SQL Injection Immunity:** සියලුම දත්ත සමුදා විමසුම් (queries) සඳහා PDO Prepared Statements සහ Bound Parameters (උදා: `:email`) පමණක් භාවිත කර ඇත. කිසිදු තැනක string concatenation නොකරයි.
- **Role-Based Access Control (RBAC):** පරිශීලකයා `admin` ද සාමාන්‍ය `user` කෙනෙක්ද යන්න හඳුනාගෙන, admin-only endpoints වලදී අනවසර පිවිසුම් (unauthorized access) වලක්වයි.

මූලික කාර්යය: පාරිභෝගිකයාට අවශ්‍ය පරිදි ඇඳුම් මැසීම (Custom tailoring) සඳහා ඉල්ලීම් භාරගැනීම, වර්ණ (Hex Color Codes) ලබාගැනීම, සහ අදියර 3ක Lifecycle එකක් ඔස්සේ Admin විසින් නිර්මාණය කර නැවත ලබාදීම.

Status Lifecycle	තේරුම සහ සිදුවන ක්‍රියාවලිය	ස්වයංක්‍රීය දැනුම්දීම (Notification)
Pending Review	පරිශීලකයා විසින් Top/Bottom වර්ණ සහ Notes සමඟ ඉල්ලීම submit කළ මොහොත.	Admin Dashboard එකේ unread badge එකක් ලෙස පෙන්වයි.
In Design	Admin විසින් ඇඳුම නිර්මාණය කිරීම ආරම්භ කර ඇති බව පාරිභෝගිකයාට දන්වයි.	"Your custom outfit request #X is now in design stage."
Ready for Download	Admin විසින් නිමකළ ඇඳුමේ ඡායාරූපය upload කර අවසන් වූ පසු.	"Your Custom Outfit (#X) is Ready for Download!"

මූලික කාර්යය: පාරිභෝගිකයාගේ Custom Requests හෝ Try-On තත්ත්වයන් පිළිබඳව alerts ලබාදීම. Unread badge counter එකක් පවත්වාගෙන යන අතර, click කළ විට `is_read = 1` ලෙස update කරයි.

මූලික කාර්යය: පරිශීලකයා විසින් මෙතෙක් අත්හදා බලන ලද 2D Try-on ප්‍රතිඵල සහ 3D Models එකම නිරයක පෙන්වීමට **LEFT JOIN** ආශ්‍රයෙන් Multi-Table Relational Data එකලස් කර ලබාදීම.

```
SELECT m.id, m.task_id, m.model_url, m.preview_image_url, m.outfit_title,
       h.result_image_url as tryon_image, g.title as garment_name
FROM user_3d_models m
LEFT JOIN try_on_history h ON m.tryon_history_id = h.id
LEFT JOIN garment g ON m.outfit_id = g.id
WHERE m.user_id = :uid ORDER BY m.id DESC;
```

මූලික කාර්යය: පරිශීලකයා උඩුගත කරන ඡායාරූපය සහ ඔවුන්ගේ පැනය (User Prompt) ලබාගෙන, MySQL database එකේ ඇති active ඇඳුම් inventory එක දත්තයක් ලෙස Gemini වෙත සපයා (Contextual Inventory Injection), පරිශීලකයාගේ ශරීර හැඩය සහ ලිංගිකත්වයට ගැළපෙන ඇඳුම් පමණක් නිර්දේශ (recommend) කිරීම.

Gender-Aware Recommendation Rule

පිරිමි පුද්ගලයෙකුගේ ඡායාරූපයක් ලබාදුන් විට, පද්ධතිය කිසිසේත්ම Bridal Gowns හෝ කාන්තා ඇඳුම් නිර්දේශ නොකරයි. `$maleItems` පමණක් prompt එකට inject කර AI එක strict constraint එකක් තුළ තබා ගනී.

මූලික කාර්යය: පරිශීලකයා කැමති ඇඳුමක වර්ණය පමණක් (උදා: Red to Royal Navy Blue) වෙනස් කිරීම සඳහා Fal.ai Flux General Inpainting microservice එක call කිරීම.

මූලික කාර්යය: Local development එකේදී PHP built-in server එක (`php -S localhost:8000 router.php`) run වන විට, CSS/JS/Images වැනි static assets නිවැරදි mime-type සමඟ serve කරමින්, API requests අදාළ backend scripts වෙත route කිරීම.

03

MySQL Database Schema & Table Structure

Style360 දත්ත සමුදාය **InnoDB Storage Engine** සහ **utf8mb4_unicode_ci** encoding මගින් සම්පූර්ණයෙන්ම Normalized ආකාරයට නිර්මාණය කර ඇත:

Table Name	ප්‍රධාන තීරු (Key Columns)	කාර්යය සහ සබඳතා (Purpose & Relationships)
users	<code>id (PK)</code> , <code>first_name</code> , <code>last_name</code> , <code>email (UNIQUE)</code> , <code>password</code> , <code>role</code>	පරිශීලක ගිණුම් තොරතුරු සහ roles (user/admin). Bcrypt passwords.
garment	<code>id (PK)</code> , <code>title</code> , <code>gender</code> , <code>category</code> , <code>display_image_url</code> , <code>fal_image_url</code> , <code>status</code>	නාමාවලියෙහි ඇති සියලුම ඇඳුම් විස්තර. Men, Women, Unisex කාණ්ඩ.

try_on_history	<code>id (PK) , user_id (FK) , garment_id (FK) , user_image_url , result_image_url , result_3d_url (TEXT)</code>	සිදුකරන ලද 2D Try-on ප්‍රතිඵල සහ 3D ආකෘති links.
custom_requests	<code>id (PK) , user_id (FK) , outfit_id (FK) , top_color , bottom_color , notes , result_image_url , status</code>	පාරිභෝගිකයාගේ bespoke tailoring ඇණවුම් සහ 3-stage status.
notifications	<code>id (PK) , user_id (FK) , title , message , is_read</code>	පරිශීලක alerts සහ completed order notifications.
user_3d_models	<code>id (PK) , user_id (FK) , task_id , image_hash (INDEX) , model_url , preview_image_url</code>	Tripo3D මගින් ජනනය කරන ලද 3D GLB Models සහ Fast Caching Hashes.

Top 15 Backend Viva Examination Questions & Model Answers

විශ්වවිද්‍යාල Viva පරීක්ෂණ මණ්ඩලය (Examiners) විසින් Backend භාර සිසුවාගෙන් ඇසීමට ඉඩ ඇති ප්‍රමුඛතම ප්‍රශ්න 15 සහ ඊට ලබාදිය යුතු විශිෂ්ටතම සිංහල හා ඉංග්‍රීසි පිළිතුරු:

Q1

ඔබේ Backend Architecture එක විස්තර කරන්න. Frontend එක සහ AI Services සමඟ Backend එක සම්බන්ධ වෙන්වේ කොහොමද?

සිංහල පිළිතුර: අපගේ Backend එක PHP 8.2 මත පදනම් වූ Stateless RESTful API Architecture එකකි. Frontend React client එකෙන් JSON හෝ Multipart payload එකක් මගින් HTTP POST/GET request එකක් එවන විට, Backend එකෙන් මුලින්ම Authentication, Session සහ Input Data validate කරනවා. ඉන්පසු දත්ත අවශ්‍ය නම් MySQL දත්ත සමුදායට (PDO මගින්) ලියන අතරතුර, Fal.ai සහ Tripo3D වැනි AI microservices සමඟ cURL ඔප්පේ secure API communication සිදුකර ප්‍රතිඵල JSON response එකක් ලෙස frontend එකට ලබාදෙනවා.

English Viva Answer: "Our backend utilizes a stateless RESTful architecture implemented in PHP 8.2. It accepts HTTPS requests from our React frontend, validates input, interacts securely with our MySQL 8.0 relational database via PDO, and orchestrates cloud AI pipelines (Fal.ai for 2D VTON, Tripo3D for 3D meshes, and Gemini Vision for validation) using asynchronous cURL streams."

Q2

SQL Injection ප්‍රහාර වලින් ඔබේ පද්ධතිය ආරක්ෂා කර ඇත්තේ කෙසේද? (Security Defense)

සිංහල පිළිතුර: අප කිසිම තැනක dynamic SQL query එකක් තුළට user input කෙලින්ම string concatenation මගින් ඇතුළත් කරන්නේ නැහැ. සියලුම database queries සඳහා 100% ක්ම **PDO Prepared Statements** සහ **Parameterized Binding** (උදා: `:email`, `:id`) පමණක් භාවිත කර තිබෙනවා. මෙහිදී query structure එක database engine එක විසින් compile කර අවසන් වන බැවින්, user input එකක් malicious SQL command එකක් ලෙස execute වීමට කිසිදු ඉඩක් නොමැත.

English Viva Answer: "We completely mitigate SQL Injection vulnerabilities by strictly adhering to PDO Prepared Statements with parameterized queries across all endpoints. User inputs are bound as pure parameter values rather than concatenated strings, preventing malicious SQL code execution."

Q3 3D Model එකක් generate කිරීමේදී API Cost එක සහ Latency එක අඩුකර ගැනීමට ඔබ කළ විශේෂ ඉංජිනේරු විසදුම කුමක්ද?

සිංහල පිළිතුර: අප `backend/generate_3d.php` හි **MD5 Content Hash Caching Algorithm** එකක් ක්‍රියාත්මක කළා. පරිශීලකයෙකු 2D image එකක් 3D කිරීමට ඉල්ලූ විට, එම image එකෙහි binary දත්ත වලින් MD5 hash එකක් සාදා `user_3d_models` වගුවේ එම hash එක තිබේදැයි බලනවා. තිබේ නම්, Tripo3D API එකට නැවත request නොයවා **තත්පර 0.03ක් (30ms)** ඇතුළත Cache එකෙන් GLB එක ලබාදෙනවා. මේ නිසා අපගේ API credits නාස්ති වීම සහ පරිශීලකයාගේ කාලය 95%කින් ඉතිරි වෙනවා.

English Viva Answer: "We engineered an MD5 cryptographic content hashing cache in generate_3d.php. Before making an expensive external API call to Tripo3D, we hash the base64 image data and query user_3d_models. If cached, it serves the existing GLB model in under 30ms, eliminating redundant cloud compute costs and user wait time."

Q4 proxy_glb.php file එකක් නිර්මාණය කිරීමට සිදුවූ තාක්ෂණික හේතුව කුමක්ද?

සිංහල පිළිතුර: කරුණු දෙකක් නිසා: පළමුවැන්න, Tripo3D CloudFront CDN එකෙන් `Access-Control-Allow-Origin: *` header එක නොඑවන විට බ්‍රවුසරයේ Three.js WebGL Loader එකට CORS error එකක් පැමිණීම. දෙවැන්න, CloudFront signed URLs පැය 24කින් expire වීමයි. `proxy_glb.php` මගින් model එක දේශීයව `temp/glb_cache/` එකෙහි cache කර, නිවැරදි CORS headers සහ binary chunks සමඟ frontend එකට stream කරයි.

English Viva Answer: "Remote CloudFront 3D assets lack permissive CORS headers needed by Three.js GLTFLoader, and signed URLs expire within 24 hours. proxy_glb.php acts as a high-performance streaming proxy that bypasses CORS restrictions and persists binary GLB files in a local cache directory for permanent availability."

Q5 Pass-through Authentication සහ Password Hashing එක සිදුවන්නේ කෙසේද?

සිංහල පිළිතුර: පරිශීලකයා ලියාපදිංචි වීමේදී (Sign Up), PHP හි `password_hash($password, PASSWORD_DEFAULT)` භාවිතයෙන් Bcrypt ඇල්ගොරිතමය ඔස්සේ 60-character cryptographically secure hash එකක් සාදා MySQL හි ගබඩා කරනවා. Login වීමේදී `password_verify($password, $dbHash)` මගින් timing attack-safe විදියට hash එක සසඳනවා. සාර්ථක වූ පසු PHP Native Sessions (`$_SESSION['user_id']`) මගින් authenticated state එක පවත්වා ගන්නවා.

English Viva Answer: "User authentication utilizes the industry-standard Bcrypt algorithm via PHP's password_hash() with automatic salt generation. Verification is handled using password_verify() to guard against timing attacks, and authenticated states are maintained via secure server-side session cookies."

Q6 Gender Mismatch එකක් (Male user selecting Female dress) Backend එකෙන් වළක්වන්නේ කෙසේද?

සිංහල පිළිතුර: `validate_gender.php` මගින් Dual-Tier Guard එකක් ක්‍රියාත්මක වෙනවා. පළමුව Regex මගින් filename පරීක්ෂා කරන අතර, දෙවනුව GD Library එකෙන් image එක 320px Thumbnail (~12KB) එකක් බවට පත්කර Google Gemini Vision API වෙත යවනවා. ලිංගිකත්වය නොගැලපේ නම්, Expensive GPU Try-On එකට යැවීමට පෙර 400 Bad Request / Mismatch error එකක් සමඟ ක්‍රියාවලිය වහාම නවත්වනවා.

English Viva Answer: "We created a dual-tier validation guard in `validate_gender.php`. It first runs a sub-millisecond regex filename check, followed by an in-memory 320px downsampled thumbnail check via Gemini Vision. If a mismatch is detected against the catalog garment's target gender, the pipeline aborts immediately before hitting costly GPU resources."

Q7 Try-on එකේදී Full-body photos වල ඔලුව හෝ දෙපා කැපීයාම (cropping) වැළැක්වූ ආකාරය පහදන්න.

සිංහල පිළිතුර: `FalService.php` හි ඇති `standardizeModelImage()` method එකෙන් Smart Aspect-Ratio Pillarboxing ඇල්ගොරිතමයක් ලියා තිබෙනවා. රූපය 9:16 portrait එකක් නම්, උස 100%ක්ම ආරක්ෂා කරමින් දෙපසට පමණක් padding එකතු කරනවා. නවද, දෙපසට කළු තීරු (black bars) නොවැටී ස්වභාවිකව පෙනීමට, රූපයේ කොනක (corner pixel) වර්ණය හඳුනාගෙන canvas එක එම පාටින්ම fill කරනවා.

English Viva Answer: "We implemented an aspect-ratio preserving pillarboxing algorithm in `FalService.php`. For tall 9:16 portraits, it preserves 100% of the vertical height from head to toe, centers the subject horizontally, and seamlessly samples the corner pixel color to fill borders without adding artificial black bars."

Q8 Face Restoration Weight එක 0.5 ට වෙනස් කිරීමට හේතුව කුමක්ද?

සිංහල පිළිතුර: Fal.ai හි CodeFormer face restoration weight එක Default 1.0 තිබෙන විට, සම්පූර්ණ ඇඟට සාපේක්ෂව හිස පමණක් කෘතීමව විශාල වී (oversized cartoonish look) පෙනුණා. අප එය `DEFAULT_FACE_RESTORATION_WEIGHT = 0.5` ලෙස සකසා ස්වභාවික මිනිස් ශරීර සමානුපාතිකත්වය (natural head-to-body ratio) ලබාගත්තා.

English Viva Answer: "The default CodeFormer face restoration weight of 1.0 caused disproportional head enlargement on full-body models. We fine-tuned the weight constraint to 0.5 in `FalService.php`, achieving photorealistic facial clarity while preserving realistic anatomical proportions."

Q9 Bespoke Tailoring (Custom Outfit Request) එකක Backend Lifecycle එක කුමක්ද?

සිංහල පිළිතුර: `custom_requests.php` මගින් අදියර 3ක Lifecycle එකක් පාලනය වේ: (1) **Pending Review:** පාරිභෝගිකයා top/bottom පාට සහ notes සමඟ request එක දමයි. (2) **In Design:** Admin ඉල්ලීම භාරගෙන මැසීම/නිර්මාණය ආරම්භ කරයි. (3) **Ready for Download:** Admin නිමකළ design image එක upload කළ පසු පාරිභෝගිකයාගේ notification center එකට ස්වයංක්‍රීය alert එකක් යයි.

English Viva Answer: "The bespoke tailoring module follows a strict 3-stage state machine: 'Pending Review' upon submission, 'In Design' when assigned to the tailoring team, and 'Ready for Download' once the completed outfit render is uploaded, triggering real-time user notification records."

Q10 Multimodal AI Stylist එකේදී Hallucinations සහ වැරදි ඇදුම් නිර්දේශ වීම වැළැක්වූයේ කෙසේද?

සිංහල පිළිතුර: **Contextual Inventory Injection** ක්‍රමවේදය මගිනි. `chat_multimodal.php` මගින් ප්‍රථමයෙන් MySQL `garment` table එකෙන් සැබවින්ම stock එකේ ඇති active ඇඳුම් ලැයිස්තුව fetch කර, user's gender එකට පමණක් match වන items Gemini Vision prompt එකට system context එකක් ලෙස inject කරයි. එම නිසා catalog එකේ නැති ඇඳුම් නිර්දේශ කිරීමෙන් AI එක වැළකේ.

English Viva Answer: "We use Contextual Inventory Injection in chat_multimodal.php. The backend fetches active garments directly from MySQL, filters them by the user's detected gender, and embeds them into the LLM system prompt, constraining Gemini to only recommend items currently in inventory."

Q11

3D Mesh එකක් හැඳීමේදී studio background එක (බිත්ති, දොරවල්) Mesh එකට එකතු වීම වැළැක්වූයේ කෙසේද?

සිංහල පිළිතුර: `generate_3d.php` හිදී ඡායාරූපය Tripo3D වෙත යැවීමට ප්‍රථමයෙන්, Fal Background Removal AI මගින් background එක සම්පූර්ණයෙන්ම ඉවත් කර, පුද්ගලයා සහ ඇඳුම පමණක් සහිත Transparent PNG එකක් සාදා Tripo3D API එකට යවනවා. එමගින් 3D mesh එක පිරිසිදු human mesh එකක් ලෙස පමණක් හැඳෙනවා.

English Viva Answer: "Before submitting to Tripo3D, `generate_3d.php` invokes an automated background removal microservice. This strips walls, floors, and studio clutter, supplying Tripo3D with an isolated transparent subject to prevent background geometry distortions."

Q12

Server එකෙහි Database Connection එක හැසිරවීමේදී Failover resilience ලබාදුන්නේ කෙසේද?

සිංහල පිළිතුර: `database.php` හි `$hostsToTry = [$host, "127.0.0.1", "localhost", "127.0.0.1;port=" . $port]` array loop එකක් ක්‍රියාත්මක වෙනවා. එක් host configuration එකක් fail වුවහොත් ඊළඟ host එකට try කරන අතර, database එක නැතිනම් root permissions මගින් `CREATE DATABASE IF NOT EXISTS style360_v2` ක්‍රියාත්මක කර reconnect වෙනවා.

English Viva Answer: "database.php implements a multi-host failover loop across localhost, loopback IP, and custom ports. If the style360_v2 database is missing, it catches the PDOException, auto-provisions the database schema using root credentials, and establishes a persistent UTF-8 connection."

Q13

3D Models වල දිගු URLs (>900 chars) Database එකේ Save කිරීමේදී ආප්‍ර Issue එක විසඳුවේ කෙසේද?

සිංහල පිළිතුර: සාමාන්‍ය `VARCHAR(255)` එකක AWS CloudFront signed query parameters සහිත URLs truncate වී data corruption ඇති විය. `database.php` හි auto-migration එකක් මගින් `ALTER TABLE try_on_history MODIFY COLUMN result_3d_url TEXT` ලෙස වෙනස් කර අසීමිත දිගකින් යුත් URLs ආරක්ෂිතව ගබඩා කිරීමට ඉඩ සැලැස්සුවා.

English Viva Answer: "CloudFront signed 3D model URLs frequently exceed 900 characters due to authentication tokens. We resolved this via automated schema migrations that upgraded result_3d_url from VARCHAR to TEXT in MySQL, preventing data truncation."

Q14

Backend එකෙහි Error Handling සහ HTTP Status Codes කළමනාකරණය කර ඇත්තේ කෙසේද?

සිංහල පිළිතුර: සියලුම endpoints හි `http_response_code()` භාවිතයෙන් RESTful සම්මුතීන්ට අනුකූලව status codes ලබාදෙනවා: සාර්ථක ඉල්ලීම් සඳහා `200 OK`, වැරදි input දත්ත සඳහා `400 Bad Request`, Database හෝ Cloud API failures සඳහා `500 Internal Server Error`, සහ pre-flight check සඳහා `OPTIONS 200 OK`. සියලුම error responses `{"status": "error", "message": "..."}` ලෙස JSON format එකෙන් නිකුත් වේ.

English Viva Answer: "Every endpoint enforces standard HTTP status codes: 200 OK for successful executions, 400 Bad Request for validation errors, and 500 for database or API exceptions. All responses maintain a uniform JSON envelope with status and descriptive message fields."

Q15

අනාගතයේදී මෙම Backend එක Enterprise Scale එකට ගෙන යාමට ඔබ යෝජනා කරන වැඩිදියුණු කිරීම් (Future Improvements) මොනවාද?

සිංහල පිළිතුර:

- 1. Redis / RabbitMQ Queue System:** AI Try-On සහ 3D Generation දිගු කාලීන (long-running) කාර්යයන් බැවින් Background Worker Queues (Celery/RabbitMQ) මඟින් asynchronous job processing කිරීම.
- 2. JWT (JSON Web Tokens):** Stateless microservice scaling සඳහා PHP session වෙනුවට JWT Bearer tokens භාවිතය.
- 3. Cloudflare R2 / AWS S3 Integration:** Local disk storage වෙනුවට direct S3 multi-region object storage භාවිතය (දැනටමත් `r2_storage.php` architecture එක ඔප්පු කර ඇත).

English Viva Answer: "For enterprise scaling, we propose: 1) Asynchronous background job workers using Redis queues to decouple long AI generation tasks, 2) Migrating session-based auth to stateless JWT bearer tokens, and 3) Offloading local file storage to Cloudflare R2 / S3 object buckets, which we have prototyped in `r2_storage.php`."



Viva Defense Tips for the Backend Candidate

- Confidence on Code:** "මම Backend එකේ Database Resilience, Caching, සහ AI Orchestration එක මුල සිටම optimize කරපු කෙනෙක්" යන ආත්ම විශ්වාසයෙන් කතා කරන්න.
- Emphasize Engineering Decisions:** නිකන්ම API එකක් call කළා කියනවාට වඩා "MD5 Caching එකෙන් 30ms latency එකක් ගන්නා", "GD In-memory downsampling එකෙන් payload එක 12KB කරලා Gemini speed එක 1.2s දක්වා වැඩි කළා", "CORS සහ Expired Signed URLs bypass කරන්න Proxy streaming හැඳුවා" යනුවෙන් technical ගැඹුර පැහැදිලි කරන්න.